shreya cu24250018

```
import os
from PIL import Image
import numpy as np # Import numpy for data type information
```

q1

```
def get_image_properties(image_path):

    try:
        # Check if file exists
        if not os.path.exists(image_path):
            print(f"Error: File not found at '{image_path}'")
            return

        # Open the image using Pillow
        img = Image.open(image_path)

        # Get dimensions
        dimensions = img.size # Returns (width, height)
        width, height = dimensions

        # Get number of channels
        # Pillow modes: '1', 'L', 'P', 'RGB', 'RGBA', 'CMYK', 'YCbCr', 'LAB', 'HSV', 'I', 'F'
        # Approximate channels based on mode
        if img.mode in ['1', 'L', 'P']:
            num_channels = 1
        elif img.mode in ['RGB', 'YCbCr', 'LAB', 'HSV']:
            num_channels = 3
        elif img.mode == 'RGBA':
            num_channels = 4
        elif img.mode == 'CMYK':
            num_channels = 4 # Or 3 depending on interpretation for display
        elif img.mode in ['I', 'F']:
            num_channels = 1 # Integer or float 32-bit pixels
        else:
            num_channels = len(img.getbands())

        # Get data type (approximated from Pillow mode and converted to numpy array)
        # Convert to numpy array to get dtype, as PIL image objects don't directly expose it
        img_array = np.array(img)
        data_type = img_array.dtype

        # Get file size
        file_size = os.path.getsize(image_path)

        print(f"--- Image Properties for '{image_path}' ---")
        print(f"Dimensions: {width}x{height} pixels")
        print(f"Number of Channels: {num_channels}")
        print(f"Data Type: {data_type}")
        print(f"File Size: {file_size} bytes")

    except FileNotFoundError:
        print(f"Error: The file at '{image_path}' was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")
```

```
# Create a dummy image for demonstration
dummy_image_path = 'dummy_image.png'
dummy_img = Image.new('RGB', (200, 150), color = 'red')
dummy_img.save(dummy_image_path)

# Now, call the function with the dummy image path
get_image_properties(dummy_image_path)

# Clean up the dummy image
os.remove(dummy_image_path)
print(f"\nCleaned up: '{dummy_image_path}'")
```

```
--- Image Properties for 'dummy_image.png' ---
Dimensions: 200x150 pixels
```

```
Number of Channels: 3
Data Type: uint8
File Size: 461 bytes

Cleaned up: 'dummy_image.png'
```

```
def convert_and_compare_grayscale(image_path, output_path='compared_image.png'):
```

q2

```
# Create a new dummy color image for demonstration
test_color_image_path = 'test_color_image.png'
test_img = Image.new('RGB', (300, 200), color = (100, 150, 200)) # A shade of blue
test_img.save(test_color_image_path)

# Define the output path for the comparison image
comparison_output_path = 'grayscale_comparison.png'

# Call the function to convert and compare
convert_and_compare_grayscale(test_color_image_path, comparison_output_path)

# Clean up the dummy images
os.remove(test_color_image_path)
os.remove(comparison_output_path)
print(f"\nCleaned up: '{test_color_image_path}' and '{comparison_output_path}'")
```

```
Original and grayscale images saved side by side to 'grayscale_comparison.png'
```



```
Cleaned up: 'test_color_image.png' and 'grayscale_comparison.png'
```

Q3

```
def convert_and_compare_formats(image_path, base_name='converted_image'):
    """
    Loads an image, converts it to JPG, PNG, and BMP formats,
    and then compares the file sizes of the resulting images.

    Args:
        image_path (str): The path to the input image file.
        base_name (str): The base name for the output files (e.g., 'converted_image').
    """
    try:
        if not os.path.exists(image_path):
            print(f"Error: Input image not found at '{image_path}'")
            return

        img = Image.open(image_path).convert('RGB') # Ensure consistent mode for saving

        output_files = {}

        # Save as JPG
        jpg_path = f"{base_name}.jpg"
        img.save(jpg_path, format='JPEG')
        output_files['JPG'] = jpg_path

        # Save as PNG
        png_path = f"{base_name}.png"
        img.save(png_path, format='PNG')
        output_files['PNG'] = png_path

        # Save as BMP
        bmp_path = f"{base_name}.bmp"
```

```
            img.save(bmp_path, format='BMP')
            output_files['BMP'] = bmp_path

            print(f"--- File Sizes for '{image_path}' Conversions ---")
            for fmt, path in output_files.items():
                file_size = os.path.getsize(path)
                print(f"{fmt}: {file_size} bytes")

            print("\nGenerated files:")
            for path in output_files.values():
                print(f"- {path}")

            # Clean up generated files
            for path in output_files.values():
                os.remove(path)
            print("\nCleaned up generated files.")

        except FileNotFoundError:
            print(f"Error: The file at '{image_path}' was not found.")
        except Exception as e:
            print(f"An error occurred during format conversion: {e}")
```

```
# Create a dummy image for format comparison
comparison_test_image_path = 'compare_formats_input.png'
dummy_img_formats = Image.new('RGB', (400, 300), color = (50, 200, 100)) # A shade of green
dummy_img_formats.save(comparison_test_image_path)

# Call the function to convert and compare formats
convert_and_compare_formats(comparison_test_image_path, 'my_image')

# Clean up the original dummy input image
os.remove(comparison_test_image_path)
print(f"\nCleaned up: '{comparison_test_image_path}'")
```

```
--- File Sizes for 'compare_formats_input.png' Conversions ---
JPG: 2528 bytes
PNG: 1066 bytes
BMP: 360054 bytes

Generated files:
- my_image.jpg
- my_image.png
- my_image.bmp

Cleaned up generated files.

Cleaned up: 'compare_formats_input.png'
```

Q4

```
from PIL import ImageOps # Import ImageOps for image manipulation

def generate_and_compare_negative(image_path, output_path='negative_comparison.png'):

    try:
        if not os.path.exists(image_path):
            print(f"Error: Input image not found at '{image_path}'")
            return

        # Open the original image and ensure it's in RGB mode for consistent negation
        original_img = Image.open(image_path).convert('RGB')

        # Generate the negative image
        negative_img = ImageOps.invert(original_img)

        # Create a new image to hold both side by side
        combined_width = original_img.width * 2
        combined_height = original_img.height
        combined_img = Image.new('RGB', (combined_width, combined_height))

        # Paste the original image on the left
        combined_img.paste(original_img, (0, 0))

        # Paste the negative image on the right
        combined_img.paste(negative_img, (original_img.width, 0))
```

```
            # Save the combined image
            combined_img.save(output_path)
            print(f"Original and negative images saved side by side to '{output_path}'")

            # Optionally display the image in Colab
            from IPython.display import display
            display(combined_img)

        except FileNotFoundError:
            print(f"Error: The file at '{image_path}' was not found.")
        except Exception as e:
            print(f"An error occurred during negative generation: {e}")
```

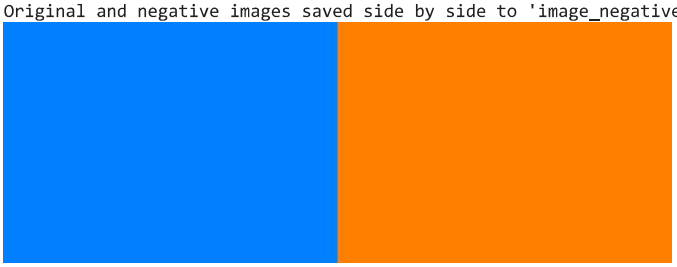
```
# Create a dummy image for negative generation demonstration
negative_test_image_path = 'negative_test_image.png'
dummy_img_negative = Image.new('RGB', (250, 180), color = (0, 128, 255)) # A shade of blue
dummy_img_negative.save(negative_test_image_path)

# Define the output path for the comparison image
negative_output_path = 'image_negative_comparison.png'

# Call the function to generate and compare the negative
generate_and_compare_negative(negative_test_image_path, negative_output_path)

# Clean up the dummy images
os.remove(negative_test_image_path)
os.remove(negative_output_path)
print(f"\nCleaned up: '{negative_test_image_path}' and '{negative_output_path}'")
```

```
Original and negative images saved side by side to 'image_negative_comparison.png'
```



```
Cleaned up: 'negative_test_image.png' and 'image_negative_comparison.png'
```

Q5

```
def batch_image_resizer(input_folder, output_folder, max_size, quality=90):

    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
        print(f"Created output folder: '{output_folder}'")

    processed_count = 0
    for filename in os.listdir(input_folder):
        if filename.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.gif')):
            input_path = os.path.join(input_folder, filename)
            output_path = os.path.join(output_folder, filename)

            try:
                img = Image.open(input_path)
                original_width, original_height = img.size

                # Calculate new dimensions preserving aspect ratio
                if original_width > original_height:
                    new_width = max_size
                    new_height = int(original_height * (max_size / original_width))
                else:
                    new_height = max_size
                    new_width = int(original_width * (max_size / original_height))

                resized_img = img.resize((new_width, new_height), Image.LANCZOS)

                # Determine format and save
                file_extension = filename.split('.')[-1].lower()
                if file_extension in ['jpg', 'jpeg']:
                    resized_img.save(output_path, format='JPEG', quality=quality)
                elif file_extension == 'png':
```

```
                    resized_img.save(output_path, format='PNG')
                elif file_extension == 'bmp':
                    resized_img.save(output_path, format='BMP')
                elif file_extension == 'gif':
                    resized_img.save(output_path, format='GIF')
                else:
                    # Fallback for unknown/unhandled formats, save as PNG
                    resized_img.save(output_path, format='PNG')

                print(f"Resized '{filename}' to {new_width}x{new_height} and saved to '{output_path}'")
                processed_count += 1

            except Exception as e:
                print(f"Error processing '{filename}': {e}")

    if processed_count == 0:
        print(f"No images found or processed in '{input_folder}'.")
    else:
        print(f"\nBatch resizing complete. Processed {processed_count} images.")
```

```
# Define input and output folders
input_dir = 'input_images'
output_dir = 'output_resized_images'
max_dimension = 150

# Create input folder if it doesn't exist
if not os.path.exists(input_dir):
    os.makedirs(input_dir)
    print(f"Created input folder: '{input_dir}'")

# Create some dummy images in the input folder
dummy_img1 = Image.new('RGB', (600, 400), color = 'purple')
dummy_img1.save(os.path.join(input_dir, 'image1.png'))

dummy_img2 = Image.new('RGB', (300, 500), color = 'orange')
dummy_img2.save(os.path.join(input_dir, 'image2.jpg'))

dummy_img3 = Image.new('RGB', (720, 720), color = 'cyan')
dummy_img3.save(os.path.join(input_dir, 'image3.bmp'))

print(f"Created dummy images in '{input_dir}'")

# Run the batch resizer
batch_image_resizer(input_dir, output_dir, max_dimension)

# --- Verification and Cleanup ---
print("\nVerifying resized images in output folder:")
if os.path.exists(output_dir):
    for filename in os.listdir(output_dir):
        img_path = os.path.join(output_dir, filename)
        if os.path.isfile(img_path):
            img = Image.open(img_path)
            print(f"  '{filename}': {img.size[0]}x{img.size[1]} pixels")
            img.close() # Close image after use

# Clean up dummy folders and images
import shutil
if os.path.exists(input_dir):
    shutil.rmtree(input_dir)
    print(f"Cleaned up input folder: '{input_dir}'")
if os.path.exists(output_dir):
    shutil.rmtree(output_dir)
    print(f"Cleaned up output folder: '{output_dir}'")
```

```
Created input folder: 'input_images'
Created dummy images in 'input_images'
Created output folder: 'output_resized_images'
Resized 'image2.jpg' to 90x150 and saved to 'output_resized_images/image2.jpg'
Resized 'image1.png' to 150x100 and saved to 'output_resized_images/image1.png'
Resized 'image3.bmp' to 150x150 and saved to 'output_resized_images/image3.bmp'

Batch resizing complete. Processed 3 images.

Verifying resized images in output folder:
  'image2.jpg': 90x150 pixels
  'image1.png': 150x100 pixels
  'image3.bmp': 150x150 pixels
```

```
Cleaned up input folder: 'input_images'
Cleaned up output folder: 'output_resized_images'
```

Q6: Image Cropper - Extracting Region of Interest

```
def crop_image_region(image_path, left, upper, right, lower, output_path='cropped_image.png'):

    try:
        if not os.path.exists(image_path):
            print(f"Error: Input image not found at '{image_path}'")
            return

        # Open the image
        img = Image.open(image_path)

        # Define the cropping box (left, upper, right, lower)
        crop_box = (left, upper, right, lower)

        # Crop the image
        cropped_img = img.crop(crop_box)

        # Save the cropped image
        cropped_img.save(output_path)
        print(f"Image successfully cropped and saved to '{output_path}'")

        # Optionally display the cropped image
        from IPython.display import display
        print("\nDisplaying the cropped image:")
        display(cropped_img)

    except FileNotFoundError:
        print(f"Error: The file at '{image_path}' was not found.")
    except Exception as e:
        print(f"An error occurred during image cropping: {e}")
```

```
# Create a dummy image for cropping demonstration
original_crop_image_path = 'original_image_for_crop.png'
dummy_img_crop = Image.new('RGB', (400, 300), color=(100, 200, 150)) # A greenish color
dummy_img_crop.save(original_crop_image_path)

print(f"Created a dummy image: '{original_crop_image_path}' ({dummy_img_crop.size[0]}x{dummy_img_crop.size[1]})\n")

# Define the region of interest (e.g., a central square)
# (left, upper, right, lower)
# For a 400x300 image, a central 100x100 square could be:
# left = (400 - 100) / 2 = 150
# upper = (300 - 100) / 2 = 100
# right = 150 + 100 = 250
# lower = 100 + 100 = 200

crop_left = 150
crop_upper = 100
crop_right = 250
crop_lower = 200

cropped_output_path = 'cropped_region.png'

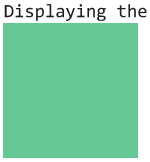
# Call the function to crop the image
crop_image_region(
    image_path=original_crop_image_path,
    left=crop_left,
    upper=crop_upper,
    right=crop_right,
    lower=crop_lower,
    output_path=cropped_output_path
)

# Clean up the dummy images
os.remove(original_crop_image_path)
os.remove(cropped_output_path)
print(f"\nCleaned up: '{original_crop_image_path}' and '{cropped_output_path}'")
```

```
Created a dummy image: 'original_image_for_crop.png' (400x300)

Image successfully cropped and saved to 'cropped_region.png'

Displaying the cropped image:
```



```
Cleaned up: 'original_image_for_crop.png' and 'cropped_region.png'
```

Q7: RGB Channel Splitter, Reconstructor, and Verifier

```
def process_rgb_channels(image_path):

    try:
        if not os.path.exists(image_path):
            print(f"Error: Input image not found at '{image_path}'")
            return

        # Open the image and ensure it's in RGB mode
        original_img = Image.open(image_path).convert('RGB')
        print(f"Original image loaded: {image_path} ({original_img.size[0]}x{original_img.size[1]})\n")

        # Split the image into individual R, G, B channels
        r_channel, g_channel, b_channel = original_img.split()
        print("Image split into Red, Green, and Blue channels.")

        # Reconstruct the image from the individual channels
        reconstructed_img = Image.merge('RGB', (r_channel, g_channel, b_channel))
        print("Image reconstructed from R, G, B channels.\n")

        # Verify pixel-level accuracy
        # Convert images to NumPy arrays for easy comparison
        original_array = np.array(original_img)
        reconstructed_array = np.array(reconstructed_img)

        if np.array_equal(original_array, reconstructed_array):
            print("Pixel-level accuracy verified: Original and reconstructed images are identical.")
        else:
            print("Pixel-level accuracy check FAILED: Original and reconstructed images differ.")

        # Optionally display the original and reconstructed images side-by-side
        from IPython.display import display
        combined_width = original_img.width * 2
        combined_height = original_img.height
        combined_display_img = Image.new('RGB', (combined_width, combined_height))
        combined_display_img.paste(original_img, (0, 0))
        combined_display_img.paste(reconstructed_img, (original_img.width, 0))

        print("\nDisplaying Original (left) and Reconstructed (right) images:")
        display(combined_display_img)

    except FileNotFoundError:
        print(f"Error: The file at '{image_path}' was not found.")
    except Exception as e:
        print(f"An error occurred during RGB channel processing: {e}")
```

```
# Create a dummy image for RGB channel processing demonstration
dummy_rgb_image_path = 'dummy_rgb_test_image.png'
dummy_rgb_img = Image.new('RGB', (350, 250), color=(70, 130, 180)) # A steel blue color
dummy_rgb_img.save(dummy_rgb_image_path)

print(f"Created a dummy image: '{dummy_rgb_image_path}'\n")

# Call the function to process RGB channels
process_rgb_channels(dummy_rgb_image_path)

# Clean up the dummy image
os.remove(dummy_rgb_image_path)
print(f"\nCleaned up: '{dummy_rgb_image_path}'")
```

```
Created a dummy image: 'dummy_rgb_test_image.png'

Original image loaded: dummy_rgb_test_image.png (350x250)

Image split into Red, Green, and Blue channels.
Image reconstructed from R, G, B channels.

Pixel-level accuracy verified: Original and reconstructed images are identical.

Displaying Original (left) and Reconstructed (right) images:
```



```
Cleaned up: 'dummy_rgb_test_image.png'
```

Q8: Image Watermarking Utility

```
from PIL import ImageDraw, ImageFont # Import for drawing text and managing fonts

def add_watermark(
    base_image_path,
    output_image_path,
    watermark_type='text', # 'text' or 'image'
    watermark_content=None, # Text string or path to watermark image
    position=(10, 10), # (x, y) coordinates for top-left corner of watermark
    opacity=0.5, # For image watermark (0.0 transparent, 1.0 opaque)
    font_path=None, # Path to a .ttf font file for text watermark
    font_size=40, # Font size for text watermark
    font_color=(0, 0, 0) # R, G, B tuple for text color (default black)
):
    """
    Adds a watermark (either text or another image) to a base image.

    Args:
        base_image_path (str): Path to the base image file.
        output_image_path (str): Path to save the watermarked image.
        watermark_type (str): Type of watermark, 'text' or 'image'.
        watermark_content (str): The text string or path to the watermark image file.
        position (tuple): (x, y) coordinates for the top-left corner of the watermark.
        opacity (float): Opacity for image watermark (0.0 to 1.0). Not used for text.
        font_path (str): Path to a TrueType font file. If None, uses a default font.
        font_size (int): Font size for text watermark.
        font_color (tuple): RGB color tuple for text watermark.
    """
    try:
        if not os.path.exists(base_image_path):
            print(f"Error: Base image not found at '{base_image_path}'")
            return

        base_img = Image.open(base_image_path).convert("RGBA") # Ensure base is RGBA for alpha blending

        if watermark_type == 'text':
            if not watermark_content:
                print("Error: Text content is required for text watermark.")
                return

            # Create a blank image for the text watermark
            txt_img = Image.new('RGBA', base_img.size, (255, 255, 255, 0)) # Transparent background
            draw = ImageDraw.Draw(txt_img)

            try:
                font = ImageFont.truetype(font_path, font_size) if font_path else ImageFont.load_default(font_size)
            except IOError:
                print(f"Warning: Font '{font_path}' not found, using default font. You can specify a system font like '/usr/sha
```

```
                font = ImageFont.load_default(font_size)
            except Exception as e:
                print(f"Error loading font: {e}, using default font.")
                font = ImageFont.load_default(font_size)

            # Calculate text size for centering or positioning
            bbox = draw.textbbox(position, watermark_content, font=font)
            text_width = bbox[2] - bbox[0]
            text_height = bbox[3] - bbox[1]

            # Apply text with font color and optional alpha (Pillow handles alpha in draw.text for RGBA images)
            draw.text(position, watermark_content, font=font, fill=font_color + (int(255 * opacity),))

            # Combine the text image with the base image
            watermarked_img = Image.alpha_composite(base_img, txt_img)

        elif watermark_type == 'image':
            if not watermark_content or not os.path.exists(watermark_content):
                print(f"Error: Watermark image not found at '{watermark_content}'")
                return

            watermark_img = Image.open(watermark_content).convert("RGBA")

            # Resize watermark if it's too big, maintaining aspect ratio
            if watermark_img.width > base_img.width / 2 or watermark_img.height > base_img.height / 2:
                watermark_img.thumbnail((base_img.width // 2, base_img.height // 2), Image.LANCZOS)

            # Create a transparent layer for the watermark
            watermark_layer = Image.new('RGBA', base_img.size, (0, 0, 0, 0))

            # Paste the watermark onto its layer
            watermark_layer.paste(watermark_img, position)

            # Apply opacity to the watermark layer
            alpha = watermark_layer.split()[3]
            alpha = Image.eval(alpha, lambda x: x * opacity) # Scale alpha values
            watermark_layer.putalpha(alpha)

            # Composite the base image and the watermark layer
            watermarked_img = Image.alpha_composite(base_img, watermark_layer)

        else:
            print("Error: Invalid watermark_type. Must be 'text' or 'image'.")
            return

        # Save the final image
        watermarked_img.save(output_image_path)
        print(f"Watermark added and saved to '{output_image_path}'")

        # Display the watermarked image
        from IPython.display import display
        print("\nDisplaying watermarked image:")
        display(watermarked_img)

    except FileNotFoundError:
        print(f"Error: A file was not found: {e}")
    except Exception as e:
        print(f"An error occurred during watermarking: {e}")
```

```
# --- Demonstration 1: Text Watermark ---
print("\n--- Demonstrating Text Watermark ---")

dummy_base_image_path_text = 'base_image_for_text_watermark.png'
dummy_base_img_text = Image.new('RGB', (500, 300), color=(255, 255, 200)) # Light yellow background
dummy_base_img_text.save(dummy_base_image_path_text)

watermarked_text_output_path = 'watermarked_image_text.png'

# Example of a common font path in Colab/Linux systems
# Adjust this if you know a specific font file you want to use.
system_font_path = "/usr/share/fonts/truetype/liberation/LiberationMono-Regular.ttf" # A common monospaced font

add_watermark(
    base_image_path=dummy_base_image_path_text,
    output_image_path=watermarked_text_output_path,
    watermark_type='text',
```

```
    watermark_content='SHREYA',
    position=(50, 100), # Position top-left
    font_path=system_font_path,
    font_size=60,
    font_color=(128, 128, 128), # Grey color
    opacity=0.6 # Opacity for the text
)

# Clean up
os.remove(dummy_base_image_path_text)
os.remove(watermarked_text_output_path)
print(f"\nCleaned up: '{dummy_base_image_path_text}' and '{watermarked_text_output_path}'")
```

```
--- Demonstrating Text Watermark ---
Watermark added and saved to 'watermarked_image_text.png'

Displaying watermarked image:
```




```
Cleaned up: 'base_image_for_text_watermark.png' and 'watermarked_image_text.png'
```

```
# --- Demonstration 2: Image Watermark ---
print("\n--- Demonstrating Image Watermark ---")

dummy_base_image_path_img = 'base_image_for_img_watermark.png'
dummy_base_img_img = Image.new('RGB', (600, 400), color=(150, 200, 255)) # Light blue background
dummy_base_img_img.save(dummy_base_image_path_img)

# Create a simple dummy logo image (e.g., a small red square with some transparency)
dummy_logo_path = 'dummy_logo.png'
dummy_logo = Image.new('RGBA', (150, 100), color=(255, 0, 0, 128)) # Red with 50% transparency
dummy_logo.save(dummy_logo_path)

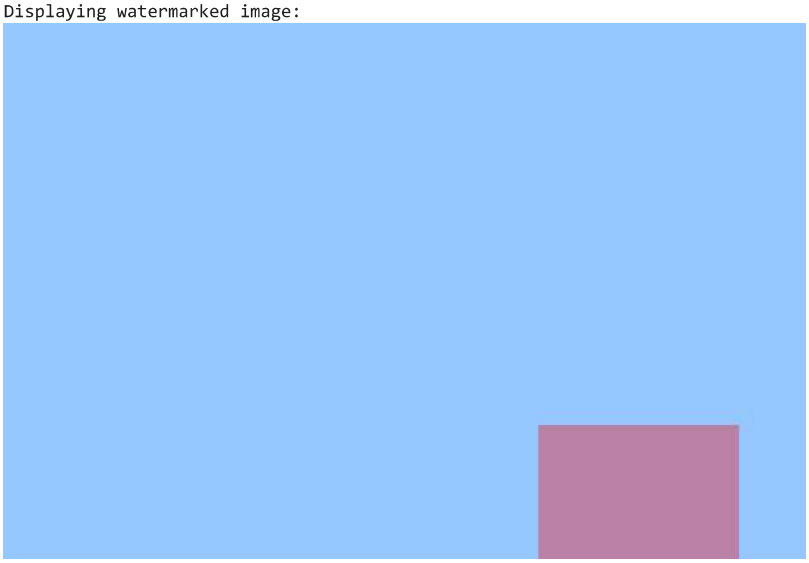
watermarked_img_output_path = 'watermarked_image_logo.png'

add_watermark(
    base_image_path=dummy_base_image_path_img,
    output_image_path=watermarked_img_output_path,
    watermark_type='image',
    watermark_content=dummy_logo_path,
    position=(400, 300), # Position bottom-right
    opacity=0.7 # Opacity for the logo
)

# Clean up
os.remove(dummy_base_image_path_img)
os.remove(dummy_logo_path)
os.remove(watermarked_img_output_path)
print(f"\nCleaned up: '{dummy_base_image_path_img}', '{dummy_logo_path}', and '{watermarked_img_output_path}'")
```

```
--- Demonstrating Image Watermark ---
Watermark added and saved to 'watermarked_image_logo.png'

Displaying watermarked image:
```



```
Cleaned up: 'base_image_for_img_watermark.png', 'dummy_logo.png', and 'watermarked_image_logo.png'
```